

LANGUAGE MODELS · CORE MECHANICS

How LLMs Work & Tokenization in Detail

A complete technical walkthrough of how large language models turn raw text into predictions — from tokenization and embeddings, through transformer attention, to training and sampling at inference time.

8

CORE CONCEPTS

3

CODE EXAMPLES

1

FULL PIPELINE

12

REVISION CARDS

02 Topic Overview

The six building blocks that make an LLM work, end to end.



Tokenization

Splitting raw text into sub-word units the model can process numerically.



Embeddings

Mapping each token ID to a dense vector that encodes meaning.



Transformer Architecture

Stacked layers of attention + feed-forward blocks that process sequences.



Self-Attention

Each token looks at every other token to decide what matters.



Training

Pretraining on huge text corpora, then fine-tuning for behavior.



Inference & Sampling

Turning output probabilities into the next chosen token, repeatedly.

03 Major Concepts

Deep explanations of each stage in the LLM pipeline.

01 Tokenization

INPUT PROCESSING

DEFINITION

Tokenization is the process of breaking raw text into smaller units called **tokens** — these can be whole words, sub-words, individual characters, or punctuation marks. Modern LLMs use **sub-word tokenization** algorithms such as **Byte-Pair Encoding (BPE)**, **WordPiece**, or **SentencePiece / Unigram**, which split rare or unknown words into smaller known pieces while keeping common words as single tokens.

WHY IT MATTERS

A model has no native concept of "letters" or "words" — it only operates on numbers. Tokenization is the bridge between human-readable text and the integer IDs a neural network can compute with. The choice of tokenizer directly affects context-window efficiency, multilingual performance, and API cost, since billing and context limits are measured in tokens, not characters or words.

KEY CONCEPTS

- **Vocabulary** — a fixed set (often 30K–200K entries) of known sub-word tokens the model can emit.
- **Byte-Pair Encoding (BPE)** — iteratively merges the most frequent adjacent character/byte pairs into new tokens.
- **Special tokens** — reserved IDs like `<bos>`, `<eos>`, `<pad>` that mark sequence boundaries.
- **Detokenization** — reversing token IDs back into human-readable text at output time.

EXAMPLE

"unbelievable" → ["un", "believ", "able"] → [403, 12891, 522]

💡 ANALOGY

Think of tokenization like packing for a trip using a fixed set of pre-made boxes instead of infinite custom containers. Common items (frequent words) get their own dedicated box. Unusual items (rare words) have to be broken apart and distributed across several smaller boxes. The model only ever "sees" and reasons about box IDs, never the raw item itself.

KEY POINT

KEY POINT

1 token $\approx$ 4 characters $\approx$ ¾ of a word in English on average — but this ratio varies significantly for other languages, code, and numbers, which is why context-window limits feel different across languages.

PYTHON • TOKENIZER EXAMPLE

```
import tiktoken

encoder = tiktoken.get_encoding("cl100k_base")

text = "Tokenization splits text into sub-words."
ids = encoder.encode(text)
tokens = [encoder.decode([i]) for i in ids]

print("Token count:", len(ids))
print("Token IDs: ", ids)
print("Tokens: ", tokens)

# Token count: 9
# Token IDs: [3404, 2065, 32307, 1495, 1139, 1207, 63550, 13]
# Tokens: ['Token', 'ization', ' splits', ' text', ' into', ' sub', '-words', '.']
```

COMMON MISTAKE

Assuming "1 word = 1 token." Long, rare, or non-English words routinely split into 3–6+ tokens, and numbers are often tokenized digit-by-digit or in odd groupings — this silently eats into context-window budgets and per-request cost if not accounted for.

02 Embeddings

REPRESENTATION

DEFINITION

An embedding is a learned mapping from a discrete token ID to a dense vector of real numbers (typically 768–12,288 dimensions). Similar tokens end up with vectors that point in similar directions in this high-dimensional space, capturing semantic and syntactic relationships learned from data.

WHY IT MATTERS

Neural networks can only perform continuous math — embeddings convert discrete symbols into a format where "distance" and "direction" become meaningful. This is what lets a model implicitly know that "king" and "queen" are related, or that "Paris" relates to "France" similarly to how "Tokyo" relates to "Japan."

KEY CONCEPTS

- **Embedding matrix** — a lookup table of shape `[vocab_size, d_model]`.
- **Positional encoding** — added to embeddings so the model knows token order.
- **Contextual vs static** — transformer embeddings become context-dependent after passing through attention layers.

EXAMPLE

```
token_id 403 ("un") → [0.021, -0.183, 0.402, ... ] (d_model dimensions)
```

03 Transformer Architecture

CORE ENGINE

DEFINITION

The transformer is a neural network architecture built from stacked identical blocks, each containing a **self-attention** sub-layer and a **feed-forward (MLP)** sub-layer, connected with residual connections and layer normalization. Most LLMs today use a **decoder-only** transformer, which predicts the next token using only preceding context.

WHY IT MATTERS

Unlike older recurrent networks that process tokens one at a time in sequence, transformers process an entire sequence in parallel, using attention instead of recurrence to model relationships between tokens. This parallelism is what made training models on internet-scale data feasible.

KEY CONCEPTS

- **Decoder-only stack** — dozens to 100+ identical transformer blocks stacked in sequence.
- **Residual connections** — each sub-layer's output is added back to its input to ease gradient flow.
- **Layer normalization** — stabilizes activations between sub-layers.
- **Feed-forward network** — a per-token MLP that adds non-linear transformation capacity.

04 Self-Attention Mechanism

CONTEXT MODELING

DEFINITION

Self-attention lets every token in a sequence compute a weighted sum over every other token, deciding how much to "attend" to each one. Each token generates a **Query (Q)**, **Key (K)**, and **Value (V)** vector; attention weights come from comparing a token's Query against every other token's Key, then those weights are used to combine the Value vectors.

WHY IT MATTERS

Attention is what lets a model resolve long-range dependencies — like matching a pronoun to the noun it refers to many sentences earlier — without the information degrading over distance the way it does in recurrent models. **Multi-head attention** runs several attention computations in parallel so the model can track different types of relationships simultaneously.

KEY CONCEPTS

- **Q, K, V matrices** — learned linear projections of the input embeddings.
- **Scaled dot-product attention** — $\text{softmax}(QK^T/\sqrt{d})V$.
- **Multi-head attention** — multiple attention "views" run in parallel, then concatenated.
- **Causal masking** — prevents a token from attending to future tokens during generation.

💡ANALOGY

Picture a crowded meeting where every participant (token) can quietly ask a targeted question (Query) to everyone else, and everyone silently offers a relevant answer (Key/Value) based on how relevant the question is to them. The participant then forms their final opinion (output) as a weighted blend of the answers that were most relevant to their question.

05 📌 Positional Encoding

SEQUENCE ORDER

DEFINITION

Because self-attention has no inherent sense of order — it treats input as a set, not a sequence — positional information must be injected explicitly. This is done via **sinusoidal** functions, **learned** position embeddings, or modern **rotary position embeddings (RoPE)** that rotate Q/K vectors based on token position.

KEY CONCEPTS

- **Absolute positional encoding** — a unique vector added per position index.
- **Relative positional encoding** — encodes the distance between token pairs instead.
- **RoPE** — widely used in modern LLMs; generalizes well to longer sequences than trained on.

06 🧠 Training: Pretraining & Fine-Tuning

LEARNING PHASE

DEFINITION

Pretraining teaches the model to predict the next token over trillions of tokens of text using self-supervised learning — no manual labels required, since the "answer" is always the next token in the corpus. **Fine-tuning** then adapts the pretrained model toward specific behaviors, typically via **Supervised Fine-Tuning (SFT)** on curated instruction/response pairs, followed by preference-based alignment such as **RLHF** or **DPO**.

WHY IT MATTERS

Pretraining is where the model acquires broad knowledge and language ability; fine-tuning is where it learns to be a helpful, instruction-following assistant rather than a raw text-completion engine. This two-stage process is why a "base model" behaves very differently from a "chat model."

KEY CONCEPTS

- **Loss function** — cross-entropy loss between predicted and actual next-token distributions.
- **SFT** — supervised fine-tuning on high-quality instruction/response examples.
- **RLHF / DPO** — aligns outputs with human preference judgments.
- **Backpropagation** — gradients flow backward through the network to update billions of weights.

07 🎲 Inference & Sampling

GENERATION

DEFINITION

At inference time, the model outputs a probability distribution (via **softmax**) over its entire vocabulary for "what comes next." A **sampling strategy** then picks the actual next token from that distribution, and the process repeats autoregressively — one token at a time — until a stop condition is reached.

KEY CONCEPTS

- **Temperature** — scales the distribution; lower = more deterministic, higher = more random.
- **Top-k sampling** — restricts sampling to the k most likely next tokens.
- **Top-p (nucleus) sampling** — samples from the smallest set of tokens whose cumulative probability exceeds p.

- **Greedy decoding** — always picks the single highest-probability token (temperature = 0).

EXAMPLE

```
P(next | "The sky is") → { "blue": 0.62, "clear": 0.14, "falling": 0.03, ... }
```

PYTHON • SAMPLING WITH TEMPERATURE

```
import torch

def sample_next_token(logits, temperature=0.7, top_p=0.9):
    logits = logits / temperature
    probs = torch.softmax(logits, dim=-1)

    sorted_probs, sorted_idx = torch.sort(probs, descending=True)
    cumulative = torch.cumsum(sorted_probs, dim=-1)

    # keep smallest set of tokens with cumulative prob <= top_p
    mask = cumulative <= top_p
    mask[0] = True

    filtered = sorted_probs[mask]
    filtered = filtered / filtered.sum()
    choice = torch.multinomial(filtered, num_samples=1)

    return sorted_idx[choice]
```

⚠️ COMMON MISTAKE

Setting temperature to 0 does not make output "fully deterministic" across all systems — batching, hardware floating-point non-determinism, and mixture-of-experts routing can still introduce small variation. Also, high temperature without top-k/top-p can sample low-probability "garbage" tokens.

08 Context Window & Limitations

CONSTRAINTS

DEFINITION

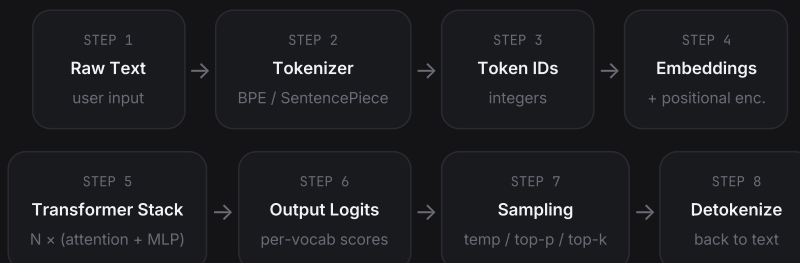
The context window is the maximum number of tokens (input + output combined) a model can process in a single request — commonly 8K to 1M+ tokens depending on the model. Attention computation scales quadratically with sequence length in the standard transformer, which is a key reason very long context windows are expensive to support.

KEY CONCEPTS

- **KV cache** — cached Key/Value vectors from previous tokens, reused to speed up generation.
- **Quadratic attention cost** — doubling sequence length roughly quadruples attention compute.
- **Context rot** — models can lose track of details buried deep in very long contexts.

04 Workflow: End-to-End Pipeline

How a single request travels from raw text to generated text.



05 Step-by-Step: Building a Tiny BPE Tokenizer

A simplified implementation to see the core algorithm in action.

1 Start with characters

Initialize the vocabulary as every individual character/byte in the training corpus. Represent each word as a sequence of these base symbols, e.g. "low" → ['l', 'o', 'w'] .

2 Count adjacent pairs

Scan the whole corpus and count how often each pair of adjacent symbols appears together, e.g. ('l', 'o') , ('o', 'w') .

3 Merge the most frequent pair

Replace every occurrence of the single most frequent pair with a new merged symbol, and add that symbol to the vocabulary.

4 Repeat until target vocab size

Continue counting and merging pairs for a fixed number of iterations (or until the vocabulary reaches its target size, e.g. 50,000 tokens).

5 Save merges as the tokenizer

The ordered list of merge rules becomes the trained tokenizer — encoding new text means applying those same merges in the same order.

PYTHON • MINIMAL BPE TRAINING LOOP

```
from collections import Counter, defaultdict

def get_pair_counts(corpus):
    pairs = Counter()
    for word, freq in corpus.items():
        symbols = word.split()
        for i in range(len(symbols) - 1):
            pairs[(symbols[i], symbols[i + 1])] += freq
    return pairs

def merge_pair(pair, corpus):
    merged = "".join(pair)
    new_corpus = {}
    for word, freq in corpus.items():
        new_word = word.replace(" ".join(pair), merged)
        new_corpus[new_word] = freq
    return new_corpus

corpus = {"l o w": 5, "l o w e r": 2, "n e w e s t": 6}
merges = []

for step in range(10):
    pairs = get_pair_counts(corpus)
    if not pairs:
        break
    best = max(pairs, key=pairs.get)
    corpus = merge_pair(best, corpus)
    merges.append(best)

print("Learned merges:", merges)
```

KEY POINT

Production tokenizers (GPT, Claude, Llama) work at the **byte level** rather than the Unicode-character level, which guarantees every possible input — including emoji, rare scripts, and malformed text — can always be encoded without an "unknown token" fallback.

06 Comparison: Tokenization Algorithms

ALGORITHM	USED BY	MERGE STRATEGY	NOTES
BPE	GPT family, many OSS models	Merges most frequent adjacent pair	Simple, fast, byte-level variants avoid unknown tokens
WordPiece	BERT family	Merges pair maximizing likelihood gain	Slightly more "linguistically aware" than raw frequency
Unigram / SentencePiece	T5, Llama, many multilingual models	Prunes a large candidate vocab via probability	Strong for multilingual and whitespace-agnostic text

07 Final Summary

What to remember

✓ Text is broken into sub-word **tokens** via BPE/WordPiece/Unigram before a model can process it.

✓ Stacked **self-attention + feed-forward** layers process the whole sequence in parallel.

✓ Generation is **autoregressive** — one token sampled at a time from a probability distribution.

✓ Each token maps to a dense **embedding** vector, combined with positional information.

✓ **Pretraining** teaches next-token prediction; **fine-tuning/RLHF** shapes behavior.

✓ **Context window** size and quadratic attention cost bound how much text a model can consider at once.

08 Quick Revision Cards

🔤 Tokenization

- › Sub-word units, not full words
- › BPE merges frequent pairs
- › ~4 chars ≈ 1 token (English)

🌐 Embeddings

- › Token ID → dense vector
- › Similar meaning → similar direction
- › + positional encoding added

🏗️ Transformer

- › Stacked attention + MLP blocks
- › Residuals + layer norm
- › Decoder-only for most LLMs

👁️ Attention

- › Q, K, V per token
- › $\text{softmax}(QK^T/\sqrt{d})V$
- › Causal mask blocks future tokens

🎓 Training

- › Pretrain: next-token prediction
- › SFT: instruction examples
- › RLHF/DPO: preference alignment

🎲 Sampling

- › Temperature controls randomness
- › Top-k / top-p limit candidate set
- › Greedy = always highest probability

09 Glossary / Terminology Cheat-Sheet

Vocabulary

The fixed set of sub-word tokens a tokenizer/model can produce or recognize.

d_{model}

The dimensionality of the embedding / hidden-state vectors flowing through the network.

Logits

Raw, un-normalized scores output by the model for each vocabulary token before softmax.

Softmax

A function that converts logits into a valid probability distribution summing to 1.

Autoregressive

Generating output one token at a time, feeding each prediction back in as input for the next.

Hallucination

Fluent output that is factually incorrect — a side-effect of probabilistic next-token prediction.

Parameters

The learned weights of the network — modern LLMs range from millions to trillions of them.

Inference

Running a trained model forward to produce output, as opposed to training it.

Fine-Tuning

Further training a pretrained model on a smaller, targeted dataset to shape its behavior.